PRACTICAL 6: RNN/LSTM/GRU POETRY & LYRICS GENERATION

```
# ============================================================
# PRACTICAL 6
# RNN / LSTM / GRU POETRY / LYRICS GENERATION
# CHARACTER-LEVEL TEXT GENERATION
# ============================================================

import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Embedding, SimpleRNN, LSTM, GRU, Dense
from tensorflow.keras.callbacks import EarlyStopping

# Reproducibility
np.random.seed(42)
tf.random.set_seed(42)

print("TensorFlow version:", tf.__version__)
```

```
TensorFlow version: 2.20.0
```

```
# ============================================================
# STEP 1: POETRY CORPUS
# ============================================================
#
# IMPORTANT:
# This is CHARACTER-LEVEL generation.
#
# For a real project, replace corpus_text with a large
# poetry/lyrics .txt file containing 100,000+ characters.
#
# The corpus below is intentionally repeated to make this
# demonstration sufficiently large for Colab.
# ============================================================

base_poem = """
the moon is shining softly over the quiet sea
and silver stars are waking in the evening sky
the gentle wind is moving through the trees
while waves are rising softly by the shore

the night is calm and peaceful everywhere
the world is sleeping underneath the light
a distant bird is singing in the air
and dreams are slowly growing through the night

when morning comes the golden sun appears
and paints the clouds with colors bright and warm
the flowers open after quiet hours
and birds begin their songs to greet the dawn

the fields are green beneath the morning sun
the river moves like silver through the land
the birds are singing and the day has come
while flowers dance beside the river bank

i walk alone beneath the morning sky
and watch the clouds move slowly far away
the wind is whispering softly as it passes
and birds are singing at the break of day

i hear a song within the falling rain
a quiet melody upon the roof
the drops are dancing down the window pane
and every cloud becomes a silver proof

the rain will come and rain will disappear
the clouds will pass and skies will turn to blue
and when the darkness brings a hidden fear
remember there is always light for you

love is a light that lives within the heart
a gentle flame that never fades away
although the world may sometimes make us part
true love can find another path each day
```

```
love is a river flowing to the sea
it moves through every valley and every land
it carries dreams and sets the spirit free
and leaves a gentle memory in the sand

love can be found in simple words and smiles
in quiet moments shared beneath the trees
it can remain through many distant miles
and travel softly on the evening breeze

the heart can break but it can heal again
the darkest night will slowly turn to dawn
and after every storm and every rain
a brighter day will surely still move on

do not be afraid when darkness fills the sky
for stars are always waiting in the night
and even when the clouds are passing by
the morning will return with golden light

the road is long and sometimes hard to walk
the hills are high and distant seems the goal
but every step and every quiet thought
can make a stronger and more hopeful soul

i walk beside the river in the spring
and watch the water shining in the sun
i hear the distant morning robins sing
and know another day has just begun

the river flows beneath the ancient trees
and carries fallen leaves along its way
the branches move gently with the breeze
while sunlight dances on the stream all day

the autumn leaves are falling to the ground
and golden colors fill the quiet lane
the wind is carrying every gentle sound
and clouds are gathering before the rain

the winter moon is shining cold and bright
the snow is resting softly on the hill
the village disappears into the night
and everything around becomes so still

then spring returns with flowers everywhere
the fields are green the sky is clear and wide
the birds are building nests beneath the air
and little streams are running by our side

the summer morning opens like a flower
the sunlight fills the garden with its glow
the golden hours seem to last forever
while warm and gentle winds begin to blow

a little bird is sitting on a tree
and singing softly to the morning light
its song is like a message meant for me
a promise that the world will soon be bright

the garden holds a thousand shades of green
the roses open underneath the sun
the brightest flowers that i have ever seen
are shining when the morning has begun

a rose may bloom and later fade away
a flower may be forgotten with the years
but memories can live beyond a day
and love can turn our sadness into tears

the stars may fade when morning comes at last
the moon may disappear into the blue
but every beautiful and precious past
will leave a little light behind for you

i dream of walking through a distant land
where mountains touch the clouds above the sea
where flowers grow like jewels upon the sand
and every river carries dreams to me

i dream of castles shining in the sun
with gardens filled with roses red and white
```

```
where every child can laugh and freely run
and every heart is filled with love and light

beyond the hills another road begins
beyond the road another dream may grow
and even when the journey sometimes turns
we find a place where peaceful waters flow

the ocean calls from far across the land
its waves are rising gently with the tide
the wind is carrying stories from the sand
and distant ships are moving side by side

the ocean sings a song that has no end
a song of waves and wind and distant rain
it calls the lonely traveler as a friend
and brings the wandering heart to life again

the mountains stand beneath the open sky
their snowy peaks are shining in the sun
the clouds are slowly moving high above
and every valley waits for day begun

the forest whispers secrets in the breeze
the branches move like dancers in the light
the sunlight falls between the ancient trees
and turns the quiet forest warm and bright

a thousand leaves are moving in the air
a thousand birds are singing from the trees
the world is full of beauty everywhere
when morning sunlight travels with the breeze

the evening comes with shadows soft and deep
the golden sunlight fades beyond the hill
the birds return and slowly go to sleep
and all the world around becomes so still

the moon appears above the quiet town
and silver light is shining on the street
the tired world is slowly settling down
and night is making everything complete

a candle shines beside an open door
a gentle light is glowing in the room
the quiet house is asking nothing more
while moonlight enters through the window

i remember days that passed so long ago
the faces and the voices of my friends
the places where the summer flowers grow
the memories that never truly end

some dreams are lost and some dreams remain
some disappear like shadows in the night
but hope can rise like flowers after rain
and turn a little darkness into light

the future waits beyond the distant hill
the road is open and the sky is clear
although the world may sometimes seem so still
there is a brighter morning drawing near

so walk beneath the open morning sky
and let the gentle wind move through your hair
let every little sorrow pass you by
and find a little happiness somewhere

when you are tired and cannot see the way
remember that the stars are shining still
the night will slowly disappear to day
and morning light will rise beyond the hill

there is a song within the human heart
a song of hope a song of love and peace
although the world may sometimes tear apart
the song can bring a moment of release

let every heart remember how to dream
let every soul remember how to care
let every person follow hope's bright beam
and leave a little kindness everywhere
```

```
a friend can be a light within the night
a hand to hold when roads are hard to cross
a voice that tells us everything is right
and helps us find our way when we are lost

the simple things are often most important
a smile a word a flower in the spring
a quiet walk a little song of morning
the happiness that ordinary moments bring

the world is full of little things to see
a butterfly a flower and a tree
a child who laughs beside a flowing stream
and all the simple beauty of a dream

the morning birds are singing in the trees
the river sparkles underneath the sun
the flowers dance together with the breeze
and all the world remembers spring has come

the evening stars are shining far above
the moon is looking down upon the sea
the world is filled with quietness and love
and dreams are floating gently back to me

when winter comes and covers all the ground
the trees are sleeping underneath the snow
the world becomes a quiet peaceful sound
and waits for spring to wake and start to grow

when spring arrives the sleeping flowers wake
the green leaves open underneath the sun
the rivers move and little branches shake
and birds return to sing when day has come

when summer comes the fields are full of gold
the warm wind moves across the open plain
the stories of the summer days are told
while clouds appear and disappear like rain

when autumn comes the leaves begin to fall
the golden forest changes every day
the evening wind begins to softly call
and carries summer slowly far away

every season has a song to sing
the winter moon the summer golden sun
the autumn leaves the flowers of the spring
each tells the story of the year begun

life is a journey filled with many roads
some roads are bright and some are dark at night
we carry all our memories and hopes
and search for places filled with warmth and light

we cannot know what waits beyond the door
we cannot see tomorrow's distant face
but every day can offer something more
a chance to find a new and better place

the world may change the years may quickly fly
the rivers may continue to the sea
but dreams can still remain beneath the sky
and hope can still become what we can be

i will remember every gentle song
i will remember every morning light
i will remember places all along
that made the darkest days a little bright

and when the road becomes too hard to walk
i will remember why i started there
i will remember every hopeful thought
and find the strength to continue everywhere

the stars above are shining like a dream
the moon is resting quietly tonight
the river moves beneath a silver beam
and everything is peaceful in the light

the night is not the ending of the day
```

```
it is a place where quiet dreams can grow
the darkness does not always mean delay
for morning waits beyond the falling snow

so keep your dreams and let your spirit rise
keep hope alive when everything seems wrong
for somewhere underneath the changing skies
there is a place where you will still belong

the road ahead is waiting just for you
the morning sun is shining on the way
the sky is bright the distant hills are blue
and hope is waiting for another day
"""


# ------------------------------------------------------------
# Make the corpus larger
# ------------------------------------------------------------

corpus_text = (base_poem + "\n") * 20

corpus_text = corpus_text.lower()

print("Number of characters:", len(corpus_text))
```

```
Number of characters: 198560
```

```
# ============================================================
# STEP 2: CHARACTER VOCABULARY
# ============================================================

chars = sorted(set(corpus_text))

char_to_idx = {
    char: i
    for i, char in enumerate(chars)
}

idx_to_char = {
    i: char
    for i, char in enumerate(chars)
}

vocab_size = len(chars)

print("Vocabulary size:", vocab_size)
print("Characters:", chars)
```

```
Vocabulary size: 28
Characters: ['\n', ' ', "'", 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's',
```

```
# ============================================================
# STEP 3: CREATE CHARACTER SEQUENCES
# ============================================================

SEQ_LENGTH = 100
STEP = 3

sequences = []
next_chars = []

for i in range(
    0,
    len(corpus_text) - SEQ_LENGTH,
    STEP
):

    sequences.append(
        corpus_text[
            i:i + SEQ_LENGTH
        ]
    )

    next_chars.append(
        corpus_text[
            i + SEQ_LENGTH
        ]
    )

print(
```

```
    "Number of training sequences:",
    len(sequences)
)
```

```
Number of training sequences: 66154
```

```
# ============================================================
# STEP 4: INTEGER ENCODING
# ============================================================

X = np.zeros(
    (
        len(sequences),
        SEQ_LENGTH
    ),
    dtype=np.int32
)

y = np.zeros(
    len(sequences),
    dtype=np.int32
)

for i, seq in enumerate(sequences):

    for t, char in enumerate(seq):

        X[
            i,
            t
        ] = char_to_idx[char]

    y[i] = char_to_idx[
        next_chars[i]
    ]


print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
X shape: (66154, 100)
y shape: (66154,)
```

```
# ============================================================
# STEP 5: TRAIN / VALIDATION SPLIT
# ============================================================

split_index = int(
    len(X) * 0.90
)

X_train = X[:split_index]
y_train = y[:split_index]

X_val = X[split_index:]
y_val = y[split_index:]

print(
    "Training sequences:",
    len(X_train)
)

print(
    "Validation sequences:",
    len(X_val)
)
```

```
Training sequences: 59538
Validation sequences: 6616
```

```
# ============================================================
# STEP 6: BUILD CHARACTER-LEVEL MODEL
# ============================================================

EMBEDDING_DIM = 64
RNN_UNITS = 128


def build_model(cell_type):

    model = Sequential()
```

```
    model.add(
        Input(
            shape=(SEQ_LENGTH,)
        )
    )

    model.add(
        Embedding(
            input_dim=vocab_size,
            output_dim=EMBEDDING_DIM
        )
    )

    if cell_type == "SimpleRNN":

        model.add(
            SimpleRNN(
                RNN_UNITS,
                dropout=0.2
            )
        )

    elif cell_type == "LSTM":

        model.add(
            LSTM(
                RNN_UNITS,
                dropout=0.2
            )
        )

    elif cell_type == "GRU":

        model.add(
            GRU(
                RNN_UNITS,
                dropout=0.2
            )
        )

    model.add(
        Dense(
            vocab_size,
            activation="softmax"
        )
    )

    model.compile(
        optimizer=tf.keras.optimizers.Adam(
            learning_rate=0.001
        ),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )

    return model
```

```
# ============================================================
# STEP 7: EARLY STOPPING
# ============================================================

def get_early_stopping():

    return EarlyStopping(
        monitor="val_loss",
        patience=5,
        min_delta=0.001,
        restore_best_weights=True,
        verbose=1
    )
```

```
# ============================================================
# STEP 8: TEMPERATURE SAMPLING
# ============================================================

def sample(predictions, temperature=0.45):

    predictions = np.asarray(
        predictions
    ).astype("float64")
```

```
    predictions = np.maximum(
        predictions,
        1e-8
    )

    predictions = np.log(
        predictions
    ) / temperature

    exp_predictions = np.exp(
        predictions -
        np.max(predictions)
    )

    probabilities = (
        exp_predictions /
        np.sum(exp_predictions)
    )

    return np.random.choice(
        len(probabilities),
        p=probabilities
    )
```

```
# ============================================================
# STEP 9: TEXT GENERATION
# ============================================================

def generate_text(
    model,
    seed_text,
    length=400,
    temperature=0.45
):

    seed_text = seed_text.lower()

    # Keep only known characters
    seed_text = "".join(
        char
        for char in seed_text
        if char in char_to_idx
    )

    # If seed is longer than sequence length,
    # keep only the last SEQ_LENGTH characters.
    seed_text = seed_text[-SEQ_LENGTH:]

    # Pad short seed
    if len(seed_text) < SEQ_LENGTH:

        seed_text = (
            " " *
            (SEQ_LENGTH - len(seed_text))
            + seed_text
        )

    generated = seed_text

    for _ in range(length):

        encoded = np.array(
            [
                [
                    char_to_idx[c]
                    for c in generated[-SEQ_LENGTH:]
                ]
            ],
            dtype=np.int32
        )

        predictions = model.predict(
            encoded,
            verbose=0
        )[0]

        next_index = sample(
            predictions,
            temperature
        )
```

```
            next_char = idx_to_char[
                next_index
            ]

            generated += next_char

        return generated
```

```
# ============================================================
# STEP 10: TRAIN SIMPLE RNN
# ============================================================

print("\n")
print("=" * 70)
print("TRAINING SIMPLE RNN")
print("=" * 70)

rnn_model = build_model(
    "SimpleRNN"
)

rnn_model.summary()

history_rnn = rnn_model.fit(
    X_train,
    y_train,
    validation_data=(
        X_val,
        y_val
    ),
    epochs=30,
    batch_size=128,
    callbacks=[
        get_early_stopping()
    ],
    verbose=1
)
```

```
======================================================================
TRAINING SIMPLE RNN
======================================================================
```

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 100, 64)	1,792
simple_rnn (SimpleRNN)	(None, 128)	24,704
dense (Dense)	(None, 28)	3,612

Total params: 30,108 (117.61 KB)
Trainable params: 30,108 (117.61 KB)
Non-trainable params: 0 (0.00 B)

```
Epoch 1/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 11s 16ms/step - accuracy: 0.3465 - loss: 2.2203 - val_accuracy: 0.4633 - val_loss: 1.8025
Epoch 2/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.4825 - loss: 1.7111 - val_accuracy: 0.5484 - val_loss: 1.5116
Epoch 3/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 11ms/step - accuracy: 0.5501 - loss: 1.4863 - val_accuracy: 0.6087 - val_loss: 1.3027
Epoch 4/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.5957 - loss: 1.3254 - val_accuracy: 0.6573 - val_loss: 1.1406
Epoch 5/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 11ms/step - accuracy: 0.6336 - loss: 1.2006 - val_accuracy: 0.6932 - val_loss: 1.0191
Epoch 6/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 10s 10ms/step - accuracy: 0.6591 - loss: 1.1053 - val_accuracy: 0.7297 - val_loss: 0.9129
Epoch 7/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 11ms/step - accuracy: 0.6836 - loss: 1.0266 - val_accuracy: 0.7526 - val_loss: 0.8282
Epoch 8/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.7012 - loss: 0.9618 - val_accuracy: 0.7709 - val_loss: 0.7731
Epoch 9/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 11ms/step - accuracy: 0.7172 - loss: 0.9096 - val_accuracy: 0.7922 - val_loss: 0.7085
Epoch 10/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.7310 - loss: 0.8623 - val_accuracy: 0.8083 - val_loss: 0.6615
Epoch 11/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.7413 - loss: 0.8230 - val_accuracy: 0.8182 - val_loss: 0.6184
Epoch 12/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 11ms/step - accuracy: 0.7504 - loss: 0.7863 - val_accuracy: 0.8312 - val_loss: 0.5850
Epoch 13/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.7617 - loss: 0.7536 - val_accuracy: 0.8419 - val_loss: 0.5441
Epoch 14/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.7657 - loss: 0.7339 - val_accuracy: 0.8504 - val_loss: 0.5210
Epoch 15/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.7729 - loss: 0.7110 - val_accuracy: 0.8493 - val_loss: 0.5097
Epoch 16/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.7786 - loss: 0.6881 - val_accuracy: 0.8620 - val_loss: 0.4778
Epoch 17/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 11ms/step - accuracy: 0.7869 - loss: 0.6672 - val_accuracy: 0.8708 - val_loss: 0.4470
Epoch 18/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.7902 - loss: 0.6481 - val_accuracy: 0.8688 - val_loss: 0.4453
Epoch 19/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.7957 - loss: 0.6318 - val_accuracy: 0.8804 - val_loss: 0.4160
Epoch 20/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 11ms/step - accuracy: 0.8027 - loss: 0.6116 - val_accuracy: 0.8888 - val_loss: 0.3971
Epoch 21/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.8046 - loss: 0.6025 - val_accuracy: 0.8931 - val_loss: 0.3772
Epoch 22/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 11ms/step - accuracy: 0.8094 - loss: 0.5859 - val_accuracy: 0.8931 - val_loss: 0.3730
Epoch 23/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.8110 - loss: 0.5802 - val_accuracy: 0.9037 - val_loss: 0.3597
Epoch 24/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.8132 - loss: 0.5673 - val_accuracy: 0.8983 - val_loss: 0.3509
Epoch 25/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 11ms/step - accuracy: 0.8158 - loss: 0.5615 - val_accuracy: 0.9011 - val_loss: 0.3449
Epoch 26/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.8215 - loss: 0.5445 - val_accuracy: 0.9055 - val_loss: 0.3277
Epoch 27/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.8228 - loss: 0.5385 - val_accuracy: 0.9061 - val_loss: 0.3290
Epoch 28/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.8233 - loss: 0.5321 - val_accuracy: 0.9154 - val_loss: 0.3064
Epoch 29/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.8271 - loss: 0.5234 - val_accuracy: 0.9179 - val_loss: 0.3091
Epoch 30/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 11ms/step - accuracy: 0.8311 - loss: 0.5093 - val_accuracy: 0.9184 - val_loss: 0.2955
Restoring model weights from the end of the best epoch: 30
```

```
# ============================================================
# STEP 11: SIMPLE RNN OUTPUT
# ============================================================

rnn_output = generate_text(
    rnn_model,
    "love is a",
    length=500,
    temperature=0.45
```

```
)

print("\n")
print("=" * 70)
print("SIMPLE RNN GENERATED TEXT")
print("=" * 70)

print(rnn_output)
```

```
======================================================================
SIMPLE RNN GENERATED TEXT
======================================================================
                                                                                                    love is alw
a gentle wind move through the wanding down the window pane
and every step ind rearing in the sky
and hope a plack the wanding skies
there is a song the trees
it ing steme bright
the sunlight danky the flouds and sovery glow
the gentle wind is whispering semememe whind something morning sky
and far les thet in the sea
it moves tha sleaks song
and kance semeling sills the glound somethres horing sings are moving sing
a sing the air se waits beyond the distant hill
the village disappear like si
```

```
# ============================================================
# STEP 12: TRAIN LSTM
# ============================================================

print("\n")
print("=" * 70)
print("TRAINING LSTM")
print("=" * 70)

lstm_model = build_model(
    "LSTM"
)

lstm_model.summary()

history_lstm = lstm_model.fit(
    X_train,
    y_train,
    validation_data=(
        X_val,
        y_val
    ),
    epochs=30,
    batch_size=128,
    callbacks=[
        get_early_stopping()
    ],
    verbose=1
)
```

```
======================================================================
TRAINING LSTM
======================================================================
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	(None, 100, 64)	1,792
lstm (LSTM)	(None, 128)	98,816
dense_1 (Dense)	(None, 28)	3,612

Total params: 104,220 (407.11 KB)
Trainable params: 104,220 (407.11 KB)
Non-trainable params: 0 (0.00 B)

```
Epoch 1/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 10s 12ms/step - accuracy: 0.3095 - loss: 2.3418 - val_accuracy: 0.4016 - val_loss: 1.9612
Epoch 2/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.4519 - loss: 1.7928 - val_accuracy: 0.5156 - val_loss: 1.5914
Epoch 3/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.5466 - loss: 1.4948 - val_accuracy: 0.6016 - val_loss: 1.3303
Epoch 4/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 6s 11ms/step - accuracy: 0.6145 - loss: 1.2833 - val_accuracy: 0.6622 - val_loss: 1.1408
Epoch 5/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.6637 - loss: 1.1200 - val_accuracy: 0.7090 - val_loss: 0.9851
Epoch 6/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 11ms/step - accuracy: 0.7049 - loss: 0.9882 - val_accuracy: 0.7495 - val_loss: 0.8547
Epoch 7/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 10s 10ms/step - accuracy: 0.7364 - loss: 0.8796 - val_accuracy: 0.7884 - val_loss: 0.7430
Epoch 8/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 11ms/step - accuracy: 0.7671 - loss: 0.7770 - val_accuracy: 0.8171 - val_loss: 0.6467
Epoch 9/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 10s 11ms/step - accuracy: 0.7958 - loss: 0.6950 - val_accuracy: 0.8445 - val_loss: 0.5611
Epoch 10/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.8218 - loss: 0.6135 - val_accuracy: 0.8696 - val_loss: 0.4837
Epoch 11/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.8438 - loss: 0.5440 - val_accuracy: 0.8934 - val_loss: 0.4157
Epoch 12/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 11ms/step - accuracy: 0.8633 - loss: 0.4813 - val_accuracy: 0.9093 - val_loss: 0.3606
Epoch 13/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.8817 - loss: 0.4274 - val_accuracy: 0.9293 - val_loss: 0.3033
Epoch 14/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 11ms/step - accuracy: 0.8981 - loss: 0.3792 - val_accuracy: 0.9420 - val_loss: 0.2619
Epoch 15/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 10s 10ms/step - accuracy: 0.9111 - loss: 0.3328 - val_accuracy: 0.9519 - val_loss: 0.2245
Epoch 16/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 11ms/step - accuracy: 0.9211 - loss: 0.2971 - val_accuracy: 0.9633 - val_loss: 0.1895
Epoch 17/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.9329 - loss: 0.2623 - val_accuracy: 0.9683 - val_loss: 0.1641
Epoch 18/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 11ms/step - accuracy: 0.9427 - loss: 0.2313 - val_accuracy: 0.9755 - val_loss: 0.1396
Epoch 19/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.9502 - loss: 0.2063 - val_accuracy: 0.9820 - val_loss: 0.1150
Epoch 20/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.9568 - loss: 0.1840 - val_accuracy: 0.9859 - val_loss: 0.0983
Epoch 21/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 11ms/step - accuracy: 0.9641 - loss: 0.1612 - val_accuracy: 0.9906 - val_loss: 0.0848
Epoch 22/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.9681 - loss: 0.1448 - val_accuracy: 0.9929 - val_loss: 0.0714
Epoch 23/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 11ms/step - accuracy: 0.9702 - loss: 0.1353 - val_accuracy: 0.9943 - val_loss: 0.0597
Epoch 24/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.9752 - loss: 0.1181 - val_accuracy: 0.9947 - val_loss: 0.0578
Epoch 25/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.9772 - loss: 0.1086 - val_accuracy: 0.9965 - val_loss: 0.0475
Epoch 26/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 11ms/step - accuracy: 0.9799 - loss: 0.0999 - val_accuracy: 0.9964 - val_loss: 0.0418
Epoch 27/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.9822 - loss: 0.0893 - val_accuracy: 0.9967 - val_loss: 0.0351
Epoch 28/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.9838 - loss: 0.0830 - val_accuracy: 0.9970 - val_loss: 0.0314
Epoch 29/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 11ms/step - accuracy: 0.9844 - loss: 0.0770 - val_accuracy: 0.9970 - val_loss: 0.0303
Epoch 30/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.9864 - loss: 0.0708 - val_accuracy: 0.9982 - val_loss: 0.0246
Restoring model weights from the end of the best epoch: 30
```

```
# ============================================================
# STEP 13: LSTM OUTPUT
# ============================================================

lstm_output = generate_text(
    lstm_model,
    "love is a",
    length=500,
    temperature=0.45
```

```
)

print("\n")
print("=" * 70)
print("LSTM GENERATED TEXT")
print("=" * 70)

print(lstm_output)
```

```
======================================================================
LSTM GENERATED TEXT
======================================================================
                                                                                                    love is and open buning

fo lanewing softly to the morning light
its song is like a message meant for me
a promise that the world will soon be bright

the garden holds a thousand shades of green
the roses open underneath the sun
the flowers dance together with the breeze
and all the world remembers spring has come

the evening stars are shining still
the night will slowly disappear to day
and morning light will rise beyond the hill

there is a song within the human heart
a song of hope a song of love and
```

```
# ============================================================
# STEP 14: TRAIN GRU
# ============================================================

print("\n")
print("=" * 70)
print("TRAINING GRU")
print("=" * 70)

gru_model = build_model(
    "GRU"
)

gru_model.summary()

history_gru = gru_model.fit(
    X_train,
    y_train,
    validation_data=(
        X_val,
        y_val
    ),
    epochs=30,
    batch_size=128,
    callbacks=[
        get_early_stopping()
    ],
    verbose=1
)
```

```
======================================================================
TRAINING GRU
======================================================================
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
embedding_2 (Embedding)	(None, 100, 64)	1,792
gru (GRU)	(None, 128)	74,496
dense_2 (Dense)	(None, 28)	3,612

Total params: 79,900 (312.11 KB)
Trainable params: 79,900 (312.11 KB)
Non-trainable params: 0 (0.00 B)

```
Epoch 1/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 6s 11ms/step - accuracy: 0.3500 - loss: 2.1829 - val_accuracy: 0.4616 - val_loss: 1.7582
Epoch 2/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - accuracy: 0.5175 - loss: 1.5851 - val_accuracy: 0.5872 - val_loss: 1.3622
Epoch 3/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.6100 - loss: 1.2753 - val_accuracy: 0.6675 - val_loss: 1.0947
Epoch 4/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.6747 - loss: 1.0590 - val_accuracy: 0.7285 - val_loss: 0.8983
Epoch 5/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - accuracy: 0.7242 - loss: 0.8987 - val_accuracy: 0.7721 - val_loss: 0.7517
Epoch 6/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 11ms/step - accuracy: 0.7625 - loss: 0.7694 - val_accuracy: 0.8109 - val_loss: 0.6280
Epoch 7/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - accuracy: 0.7963 - loss: 0.6620 - val_accuracy: 0.8427 - val_loss: 0.5243
Epoch 8/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - accuracy: 0.8239 - loss: 0.5722 - val_accuracy: 0.8752 - val_loss: 0.4384
Epoch 9/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 11ms/step - accuracy: 0.8482 - loss: 0.4985 - val_accuracy: 0.8989 - val_loss: 0.3689
Epoch 10/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - accuracy: 0.8700 - loss: 0.4309 - val_accuracy: 0.9169 - val_loss: 0.3097
Epoch 11/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - accuracy: 0.8884 - loss: 0.3789 - val_accuracy: 0.9309 - val_loss: 0.2663
Epoch 12/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.9014 - loss: 0.3381 - val_accuracy: 0.9427 - val_loss: 0.2268
Epoch 13/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - accuracy: 0.9137 - loss: 0.3000 - val_accuracy: 0.9531 - val_loss: 0.1960
Epoch 14/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.9232 - loss: 0.2661 - val_accuracy: 0.9602 - val_loss: 0.1686
Epoch 15/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - accuracy: 0.9322 - loss: 0.2415 - val_accuracy: 0.9643 - val_loss: 0.1439
Epoch 16/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - accuracy: 0.9381 - loss: 0.2173 - val_accuracy: 0.9729 - val_loss: 0.1260
Epoch 17/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 11ms/step - accuracy: 0.9432 - loss: 0.2008 - val_accuracy: 0.9755 - val_loss: 0.1116
Epoch 18/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - accuracy: 0.9498 - loss: 0.1822 - val_accuracy: 0.9802 - val_loss: 0.0985
Epoch 19/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - accuracy: 0.9527 - loss: 0.1688 - val_accuracy: 0.9817 - val_loss: 0.0883
Epoch 20/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 11ms/step - accuracy: 0.9564 - loss: 0.1574 - val_accuracy: 0.9831 - val_loss: 0.0792
Epoch 21/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - accuracy: 0.9612 - loss: 0.1428 - val_accuracy: 0.9872 - val_loss: 0.0684
Epoch 22/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.9632 - loss: 0.1342 - val_accuracy: 0.9841 - val_loss: 0.0686
Epoch 23/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.9633 - loss: 0.1301 - val_accuracy: 0.9873 - val_loss: 0.0616
Epoch 24/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - accuracy: 0.9668 - loss: 0.1209 - val_accuracy: 0.9885 - val_loss: 0.0572
Epoch 25/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 11ms/step - accuracy: 0.9667 - loss: 0.1164 - val_accuracy: 0.9870 - val_loss: 0.0540
Epoch 26/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - accuracy: 0.9696 - loss: 0.1090 - val_accuracy: 0.9891 - val_loss: 0.0490
Epoch 27/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - accuracy: 0.9707 - loss: 0.1035 - val_accuracy: 0.9912 - val_loss: 0.0426
Epoch 28/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 5s 11ms/step - accuracy: 0.9736 - loss: 0.0960 - val_accuracy: 0.9909 - val_loss: 0.0410
Epoch 29/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - accuracy: 0.9731 - loss: 0.0949 - val_accuracy: 0.9900 - val_loss: 0.0421
Epoch 30/30
466/466 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - accuracy: 0.9730 - loss: 0.0940 - val_accuracy: 0.9896 - val_loss: 0.0425
Restoring model weights from the end of the best epoch: 28
```

```
# ============================================================
# STEP 15: GRU OUTPUT
# ============================================================

gru_output = generate_text(
    gru_model,
    "love is a",
    length=500,
    temperature=0.45
```

```
)

print("\n")
print("=" * 70)
print("GRU GENERATED TEXT")
print("=" * 70)

print(gru_output)
```

```
======================================================================
GRU GENERATED TEXT
======================================================================
                                                                                                    love is always light for you

i dream of walking through a distant land
where mountains touch the clouds with colors bright and warm
the flowers open after quiet hours
and birds begin their songs to greet the dawn

the fields are full of gold
the warm wind moves across the open plain
the stories of the summer days are told
while clouds appear and disappear like shadows in the night
but hope can rise like flowers after rain
and turn a little darkness into light

the flowers of the spring
a quiet walk a li
```

```
# ============================================================
# STEP 16: LOSS COMPARISON
# ============================================================

print("\n")
print("=" * 70)
print("MODEL COMPARISON")
print("=" * 70)

rnn_best = min(
    history_rnn.history["val_loss"]
)

lstm_best = min(
    history_lstm.history["val_loss"]
)

gru_best = min(
    history_gru.history["val_loss"]
)

print(
    f"SimpleRNN | Best Validation Loss: {rnn_best:.4f}"
)

print(
    f"LSTM      | Best Validation Loss: {lstm_best:.4f}"
)

print(
    f"GRU       | Best Validation Loss: {gru_best:.4f}"
)
```

```
======================================================================
MODEL COMPARISON
======================================================================
SimpleRNN | Best Validation Loss: 0.2955
LSTM      | Best Validation Loss: 0.0246
GRU       | Best Validation Loss: 0.0410
```

```
# ============================================================
# STEP 17: GENERATED TEXT COMPARISON
# ============================================================

print("\n")
print("=" * 70)
print("GENERATED TEXT COMPARISON")
print("=" * 70)

print("\n--- SIMPLE RNN ---\n")
```

```
print(rnn_output)

print("\n--- LSTM ---\n")
print(lstm_output)

print("\n--- GRU ---\n")
print(gru_output)
```

```
======================================================================
GENERATED TEXT COMPARISON
======================================================================

--- SIMPLE RNN ---

                                                                                love is alw
a gentle wind move through the wanding down the window pane
and every step ind rearing in the sky
and hope a plack the wanding skies
there is a song the trees
it ing steme bright
the sunlight danky the flouds and sovery glow
the gentle wind is whispering semememe whind something morning sky
and far les thet in the sea
it moves tha sleaks song
and kance semeling sills the glound somethres horing sings are moving sing
a sing the air se waits beyond the distant hill
the village disappear like si

--- LSTM ---

                                                                                love is and open buning
fo lanewing softly to the morning light
its song is like a message meant for me
a promise that the world will soon be bright

the garden holds a thousand shades of green
the roses open underneath the sun
the flowers dance together with the breeze
and all the world remembers spring has come

the evening stars are shining still
the night will slowly disappear to day
and morning light will rise beyond the hill

there is a song within the human heart
a song of hope a song of love and

--- GRU ---

                                                                                love is always light for you

i dream of walking through a distant land
where mountains touch the clouds with colors bright and warm
the flowers open after quiet hours
and birds begin their songs to greet the dawn

the fields are full of gold
the warm wind moves across the open plain
the stories of the summer days are told
while clouds appear and disappear like shadows in the night
but hope can rise like flowers after rain
and turn a little darkness into light

the flowers of the spring
a quiet walk a li
```

```
# ============================================================
# STEP 18: LOSS CURVES
# ============================================================

plt.figure(figsize=(7, 4))

plt.plot(
    history_rnn.history["val_loss"],
    label="SimpleRNN"
)

plt.plot(
    history_lstm.history["val_loss"],
    label="LSTM"
)

plt.plot(
    history_gru.history["val_loss"],
    label="GRU"
)
```

```
plt.xlabel("Epoch")
plt.ylabel("Validation Loss")

plt.title(
    "SimpleRNN vs LSTM vs GRU"
)

plt.legend()
plt.grid(True)

plt.tight_layout()

plt.show()
```

